



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS

Anna Clara Olbi - 24015392
Bernardo Alberto Amaro - 25014832
Larissa Furlan Ferreira - 24021876
Leonardo Mateus O. Vicente - 24007335
Maria Gabriella Xavier Puccinelli - 24002540
Thiago Rubim Tranquilim - 23015590

Padrões e Arquitetura de Software

CAMPINAS

2026

1. Introdução	3
2. Atributos de Qualidade e Decisões Arquiteturais	4
2A. Atributos de Qualidade Prioritários	4
2B. Registro de Decisões Arquiteturais (ADRs)	5
3. Estilo Arquitetural	8
3A. Plano Macro: Decomposição em Unidades Implantáveis	8
3B. Plano Interno: Organização do código dentro da unidade	8
4. Aplicação dos Princípios SOLID	9
4.1 Single Responsibility Principle (SRP)	9
4.2 Open-Closed Principle (OCP)	9
4.3 Liskov Substitution Principle (LSP)	10
4.4 Interface Segregation Principle (ISP)	11
4.5 Dependency Inversion Principle (DIP)	12
5. Clean Code	14
5.1 Nomes Claros e Descritivos	14
5.2 Funções Pequenas com Responsabilidade Única	14
5.3 Ausência de Duplicação Significativa	15
5.4 Baixo Acoplamento e Alta Coesão	15
5.5 Ausência de Comentários Redundantes	16
5.6 Tratamento de Erros Explícito	16
6. Padrões de Projeto GoF	18
6.1 Strategy — Categoria: Comportamental	18
6.2 Factory Method — Categoria: Criacional	19
6.3 Prototype — Categoria: Criacional	20
6.4 Template Method — Categoria: Comportamental	22
7. Design de API	24
8. Diagramas e Modelos	25
8.1 Arquitetura C4 — Nível de Componentes	25
8.2 Diagrama de Classes — Padrões GoF	26
9. Conclusões	27
10. Referências Bibliográficas	29

1. Introdução

O Shadowhunters RPG é um jogo de texto desenvolvido em Java, ambientado no universo da série The Mortal Instruments. O jogador assume o papel de um dos quatro Shadowhunters — Clary Fairchild, Jace Herondale, Isabelle Lightwood ou Alec Lightwood — e deve percorrer cinco capítulos narrativos com escolhas, batalhas por turnos e uso estratégico de itens do inventário, com o objetivo de impedir que Valentine Morgenstern use o Cálice Mortal para criar um exército das sombras.

Objetivos gerais

- Demonstrar a aplicação de padrões de projeto GoF (Strategy, Factory Method, Prototype e Template Method) em um contexto de jogo.
- Aplicar os princípios SOLID e práticas de Clean Code de forma concreta e rastreável no código-fonte.
- Documentar decisões arquiteturais por meio de ADRs.

Objetivos específicos

- Separar responsabilidades por pacotes (monolito modular), evitando acoplamento indevido entre narrativa, batalha, domínio e utilitários.
- Tornar o sistema extensível: adicionar personagens, itens e efeitos sem modificar o núcleo existente.
- Manter o código legível e livre de duplicação, utilizando InputUtil, Console e EffectFactory.

Problema resolvido e público-alvo

O sistema resolve o desafio de estruturar um jogo de RPG textual de forma didática e extensível, sem frameworks externos. O público-alvo é acadêmico: estudantes e professores de disciplinas de engenharia de software interessados em ver padrões GoF e princípios SOLID aplicados em código Java funcional e de pequeno porte.

2. Atributos de Qualidade e Decisões Arquiteturais

2A. Atributos de Qualidade Prioritários

- **Manutenibilidade (Maintainability)**

Justificativa: O projeto é acadêmico e evolui iterativamente. A capacidade de adicionar personagens, efeitos e capítulos sem quebrar o que já existe é central para o sucesso do desenvolvimento em equipe. Código difícil de manter inviabiliza a adição de features e a correção de defeitos.

Decisões arquiteturais: Adoção do padrão Strategy para efeitos de item (cada efeito em classe separada), Factory Method para criação centralizada, Prototype para cópia segura e organização em pacotes coesos. O princípio OCP garante extensão sem modificação.

Métrica observável: Adicionar um novo tipo de efeito de item requer apenas uma nova classe implementando ItemEffect e uma entrada em EffectFactory — nenhuma classe existente de domínio precisa ser alterada.

- **Confiabilidade (Reliability)**

Justificativa: Entradas inválidas do usuário não podem derrubar o jogo. A robustez do tratamento de entrada é crítica para a experiência do jogador, que interage exclusivamente via terminal.

Decisões arquiteturais: InputUtil.lerIntervalo captura InputMismatchException e repete a leitura até receber um valor válido. Inventario lança IllegalStateException com mensagem clara quando um item sem estoque é usado. Personagem.setPontosVida garante que HP nunca fique negativo.

Métrica observável: Zero falhas por entrada inválida (texto onde se espera número, número fora do intervalo) nos fluxos de batalha, capítulo e inventário.

- **Usabilidade (Usability)**

Justificativa: Por ser um jogo de texto, toda a experiência do usuário depende da clareza das mensagens e da formatação do console. Sem

destaque visual adequado, o jogador perde o rastreamento do estado da batalha

Decisões arquiteturais: A classe Console centraliza os códigos ANSI de formatação (negrito, títulos), eliminando literais espalhados. Mensagens descritivas indicam dano, uso de item, habilidade especial e resultado da batalha. O inventário é exibido ordenado e numerado para facilitar a escolha.

Métrica observável: Toda ação do jogador recebe feedback textual imediato com indicação de HP atual, dano causado/recebido e itens obtidos.

2B. Registro de Decisões Arquiteturais (ADRs)

- **ADR-001: Adotar Monolito Modular como Arquitetura Macro**

Status: Accepted

Contexto: O jogo é executado localmente no terminal, sem requisitos de deploy distribuído, banco de dados ou APIs externas. Três grupos de responsabilidade precisavam ser separados: fluxo do jogo, regras de domínio e interação com o usuário.

Decisão: Adotar monolito modular com responsabilidades separadas por pacotes Java: chapter (narrativa), battle (combate), model (domínio), factory (criação), ui (saída) e util (entrada).

Consequências: Compilação simples sem dependências externas; separação clara entre camadas; aderência ao SRP. Risco: se o jogo evoluir para multiplayer, a arquitetura precisará ser revisada.

- **ADR-002: Usar Strategy para os Efeitos de Item**

Status: Accepted

Contexto: Na versão anterior, a classe Item concentrava toda a lógica de efeitos num switch, violando OCP e SRP.

Decisão: Adotar o padrão Strategy com a interface ItemEffect e implementações concretas: CuraEffect, CuraGrandeEffect, ForcaEffect, ElixirForcaEffect, DefesaEffect e BonusDefesaEffect.

Consequências: Item passa a delegar efeitos; novos efeitos exigem apenas nova classe. Custo: mais classes pequenas no projeto.

- **ADR-003: Centralizar Criação com Factory Method**

Status: Accepted

Contexto: Sem fábrica, o ponto de entrada e Item precisariam conhecer todas as classes concretas de personagens e efeitos, aumentando o acoplamento.

Decisão: Criar PersonagemFactory.criar(int) e EffectFactory.criar(String) como pontos únicos de instanciação.

Consequências: Código cliente depende apenas de abstrações (Personagem, ItemEffect). Custo: novos tipos devem ser registrados manualmente nas fábricas.

- **ADR-004: Usar Prototype para Clonagem de Objetos**

Status: Accepted

Contexto: Personagens possuem Inventario que contém Itens mutáveis. Cópia rasa compartilharia estado, gerando bugs ao transferir itens de inimigos derrotados.

Decisão: Implementar Prototype com Cloneable, métodos clone() e construtores de cópia em Personagem, Inventario e Item, garantindo deep copy.

Consequências: Cópia segura e encapsulada. Risco: campos mutáveis futuros precisam ser adicionados ao construtor de cópia.

- **ADR-005: Centralizar Leitura Validada em InputUtil**

Status: Accepted (decisão inicial de métodos locais foi revertida)

Contexto: A leitura de inteiros dentro de um intervalo estava duplicada em Batalha, Capítulos e ShadowhuntersRPG com pequenas inconsistências entre as cópias.

Decisão Inicial Revertida: Manter métodos locais foi descartado pois as diferenças eram acidentais, não regras de negócio distintas.

Decisão Final: Criar InputUtil.lerIntervalo(Scanner, int, int) como único ponto de leitura numérica validada em toda a aplicação.

Consequências: Eliminação de duplicação; comportamento único e consistente para entradas inválidas. Custo: mensagens de erro padronizadas, sem personalização por contexto.

3. Estilo Arquitetural

3A. Plano Macro: Decomposição em Unidades Implantáveis

O sistema adota um monolito modular: roda como único processo Java compilado com javac, sem dependências externas, servidores ou bancos de dados. Essa escolha é adequada ao escopo acadêmico e ao modo de execução (terminal local).

Um conjunto de microsserviços seria desnecessariamente complexo para um jogo single-player sem estado persistente, aumentando latência e infraestrutura sem nenhum ganho real. A arquitetura distribuída só se justificaria caso o jogo evoluísse para multiplayer online ou precisasse de backend com persistência.

A modularidade interna, expressa pelos pacotes Java, garante separação de responsabilidades equivalente ao que microsserviços oferecem em escala maior, porém sem o overhead de comunicação entre processos.

3B. Plano Interno: Organização do código dentro da unidade

Internamente, o projeto segue uma organização em camadas com ênfase em coesão por funcionalidade, aproximando-se da Clean Architecture de Robert C. Martin no que diz respeito à separação entre domínio e infraestrutura:

Pacote	Responsabilidade	Equivalente arquitetural
br.shadowhunters	Ponto de entrada e orquestração	Application Layer
chapter	Narrativa e progressão de capítulos	Application Layer
battle	Loop de combate e regras de turno	Application Layer
model	Entidades do domínio	Domain Layer
model.effect	Estratégias de efeito (Strategy GoF)	Domain Layer
factory	Criação centralizada de objetos	Domain Layer
ui	Formatação de saída no console	Interface/Infrastructure
util	Leitura validada de entrada	Interface/Infrastructure

4. Aplicação dos Princípios SOLID

4.1 Single Responsibility Principle (SRP)

Explicação técnica: Uma classe deve ter apenas um motivo para mudar — deve encapsular uma única responsabilidade.

Aplicação no projeto: A classe Batalha é responsável exclusivamente pelo loop de combate turno a turno. Ela não contém narrativa nem lógica de capítulo. A classe Console cuida apenas da formatação de saída ANSI. InputUtil cuida apenas de leitura validada. Cada uma muda por um único motivo.

```
// Console: única responsabilidade — formatação de saída
public final class Console {

    private static final String NEGRITO = "\u001B[1m";
    private static final String RESET  = "\u001B[0m";

    private Console() {}

    public static void titulo(String texto) {
        System.out.println("\n" + NEGRITO + "=== " + texto + " ===" +
RESET);
    }

    public static String emNegrito(String texto) {
        return NEGRITO + texto + RESET;
    }

}
```

Efeito: Caso a formatação do console mude (ex.: uso de uma biblioteca de cores mais rica), apenas Console precisa ser modificada, sem impacto em Batalha, Capítulos ou no domínio.

4.2 Open-Closed Principle (OCP)

Explicação técnica: Entidades de software devem estar abertas para extensão, mas fechadas para modificação.

Aplicação no projeto: A interface `ItemEffect` define o contrato de efeito. Novos efeitos são adicionados criando uma nova classe que implementa `ItemEffect`, sem modificar `Item`, `Inventario` ou nenhuma classe existente de domínio.

```
// Interface fechada para modificação

public interface ItemEffect {

    void aplicar(Personagem alvo, String nomeItem);

}
```

```
// Extensão sem modificar ItemEffect ou Item

public class ElixirForcaEffect implements ItemEffect {

    private static final int BONUS_ATAQUE = 3;

    @Override

    public void aplicar(Personagem alvo, String nomeItem) {

        alvo.setAtaque(alvo.getAtaque() + BONUS_ATAQUE);

        System.out.println("\n" + alvo.getNome() + " usou " + nomeItem

            + " e ganhou +" + BONUS_ATAQUE + " de ataque!");

    }

}
```

Efeito: O sistema de itens é extensível sem risco de regressão: cada adição de efeito é um arquivo novo, não uma modificação num arquivo já testado.

4.3 Liskov Substitution Principle (LSP)

Explicação técnica: Subtipos devem poder substituir seus tipos-base sem alterar a corretude do programa.

Aplicação no projeto: Batalha recebe um `Personagem` e um `Inimigo`. Internamente, qualquer subclasse de `Personagem` (Clary, Jace, Isabelle, Alec) pode ser passada sem que o loop de batalha precise saber qual é a concreta. O polimorfismo de `habilidadeEspecial` é o ponto central.

```
// Batalha usa apenas a abstração Personagem
public class Batalha {
    private final Personagem jogador;

    ...

    private void usarHabilidadeEspecial() {
        // Funciona corretamente para qualquer subclasse
        jogador.habilidadeEspecial(dado, inimigo);
    }
}
```

Efeito: Adicionar um quinto personagem (ex.: Simon Lewis) não requer nenhuma alteração em Batalha. A substituição é transparente.

4.4 Interface Segregation Principle (ISP)

Explicação técnica: Clientes não devem ser forçados a depender de métodos que não usam. Interfaces devem ser coesas e específicas.

Aplicação no projeto: A interface ItemEffect expõe exatamente um método — aplicar(Personagem, String). Não há métodos genéricos que forçariam implementações vazias. Comparable é implementado separadamente em cada classe que precisa de ordenação, sem obrigar todas as entidades a implementá-la.

```
// Interface enxuta — apenas o contrato necessário
public interface ItemEffect {
    void aplicar(Personagem alvo, String nomeItem);
}

// Clary implementa Comparable separado de Personagem
public class Clary extends Personagem implements Comparable<Clary> {
    @Override
    public int compareTo(Clary outro) {
        return nome.compareToIgnoreCase(outro.nome);
    }
}
```

```
}  
}
```

Efeito: Nenhuma implementação de ItemEffect precisa de método vazio ou stub — o contrato é mínimo e coeso.

4.5 Dependency Inversion Principle (DIP)

Explicação técnica: Módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de abstrações.

Aplicação no projeto: A classe Item depende da abstração ItemEffect, não das classes concretas (CuraEffect, ForcaEffect etc.). A instância concreta é injetada via EffectFactory, mantendo Item isolado das implementações.

```
public class Item implements Comparable<Item>, Cloneable {  
    private final ItemEffect efeito; // depende da abstração  
  
    public Item(String nome, String descricao,  
                String chaveEfeito, int quantidade) {  
        this.efeito = EffectFactory.criar(chaveEfeito); // fábrica resolve  
        ...  
    }  
  
    public void usar(br.shadowhunters.model.Personagem alvo) {  
        if (quantidade <= 0) {  
            throw new IllegalStateException("Não há " + nome + "  
disponível!");  
        }  
        quantidade--;  
        efeito.aplicar(alvo, nome); // polimorfismo — não sabe qual  
efeito  
    }  
}
```

Efeito: Item nunca importa nem menciona CuraEffect ou qualquer outra implementação concreta. A inversão de dependência torna o código extensível sem modificação.

5. Clean Code

5.1 Nomes Claros e Descritivos

Variáveis, métodos e classes recebem nomes que revelam intenção, eliminando a necessidade de comentários explicativos.

```
private static void exibirMenuPrincipal() {...}

private static Personagem criarPersonagem(Scanner sc) {...}

private static void popularInventarioInicial(Personagem jogador) {...}

// Constante nomeada elimina magic string duplicada

private static final String NOME_CHEFE_FINAL = "Valentine Morgenstern";
```

5.2 Funções Pequenas com Responsabilidade Única

Batalha divide seu fluxo em métodos de uma única responsabilidade, cada um com no máximo 10 linhas:

```
private void realizarAtaque() {

    int rolagemJogador = dado.nextInt(6) + 1;

    int rolagemInimigo = dado.nextInt(6) + 1;

    int danoAoInimigo = Math.max(0, jogador.getAtaque() +
rolagemJogador - inimigo.getDefesa());

    int danoAoJogador = Math.max(0, inimigo.getAtaque() +
rolagemInimigo - jogador.getDefesa());

    inimigo.receberDano(danoAoInimigo);

    jogador.receberDano(danoAoJogador);

    System.out.println("\nVocê causou " + danoAoInimigo + " de
dano!");

    System.out.println("O inimigo causou " + danoAoJogador + " de
dano!");

}
```

5.3 Ausência de Duplicação Significativa

A leitura validada de inteiros era duplicada em três classes. A ADR-005 formalizou a extração para InputUtil:

```
// Um único método para toda leitura numérica validada
public static int lerIntervalo(Scanner sc, int min, int max) {
    while (true) {
        try {
            int valor = sc.nextInt();

            if (valor < min || valor > max) {
                System.out.println("Digite um número entre " + min + " e "
+ max + ":");
                continue;
            }

            return valor;
        } catch (InputMismatchException e) {
            System.out.println("Entrada inválida - digite um número:");
            sc.next();
        }
    }
}
```

5.4 Baixo Acoplamento e Alta Coesão

Inventario expõe cópia defensiva da lista interna, preservando o encapsulamento e evitando que código externo modifique o estado interno:

```
public List<Item> getItens() {
    return new ArrayList<>(itens); // cópia defensiva
}
```

O empilhamento automático de itens iguais mantém a lógica coesa dentro de Inventario:

```
public void adicionarItem(Item item) {  
    for (Item existente : itens) {  
        if (existente.equals(item)) {  
            existente.adicionar(item.getQuantidade());  
            return;  
        }  
    }  
    itens.add((Item) item.clone());  
}
```

5.5 Ausência de Comentários Redundantes

Comentários são reservados para explicar o porquê, não o quê. Os Javadocs existentes explicam a intenção arquitetural, não o código óbvio:

```
/**  
 * Ponto de entrada do jogo Shadowhunters RPG.  
 *  
 * <p>Responsável apenas por: inicializar recursos, exibir o menu de  
seleção  
 * de personagem e orquestrar a sequência de capítulos (SRP).</p>  
 */  
public class ShadowhuntersRPG {...}  
/** Construtor de cópia usado pelo padrão Prototype. */  
public Inventario(Inventario modelo) { ... }
```

5.6 Tratamento de Erros Explícito

Erros são sinalizados com exceções significativas e mensagens claras, sem retornar valores especiais como -1 ou null:

```
// Item - estoque zerado

public void usar(br.shadowhunters.model.Personagem alvo) {

    if (quantidade <= 0) {

        throw new IllegalStateException("Não há " + nome + " disponível!");

    }

    quantidade--;

    efeito.aplicar(alvo, nome);

}

// EffectFactory - chave desconhecida
default:

    throw new IllegalArgumentException("Efeito desconhecido: " + chave);

// Inventario - modelo nulo

public Inventario(Inventario modelo) {

    if (modelo == null) throw new IllegalArgumentException("Modelo de inventário ausente");

    ...

}
```

6. Padrões de Projeto GoF

6.1 Strategy — Categoria: Comportamental

Problema motivador: Na versão anterior do jogo, a classe Item continha um switch com toda a lógica de efeitos. Adicionar um novo efeito exigia modificar Item, violando OCP e tornando a classe cada vez maior.

Implementação: A interface ItemEffect define o contrato de efeito. Cada comportamento concreto é encapsulado em sua própria classe. Item mantém uma referência para ItemEffect e delega.

```
// Contrato (Strategy)
public interface ItemEffect {
    void aplicar(Personagem alvo, String nomeItem);
}

// Estratégia concreta
public class CuraEffect implements ItemEffect {
    private static final int CURA_HP = 30;

    @Override
    public void aplicar(Personagem alvo, String nomeItem) {
        alvo.setPontosVida(alvo.getPontosVida() + CURA_HP);

        System.out.println("\n" + alvo.getNome() + " usou " + nomeItem
            + " e recuperou " + CURA_HP + " HP!");
    }
}

// Item delega - não conhece qual efeito é
public void usar(br.shadowhunters.model.Personagem alvo) {
    quantidade--;

    efeito.aplicar(alvo, nome); // Strategy em ação
}
```

Benefício: Adicionar VenenoEffect ou TeleporteEffect não toca em Item, Inventario nem em nenhum efeito existente. Custo: Maior número de classes pequenas; EffectFactory precisa ser atualizada a cada novo efeito.

6.2 Factory Method — Categoria: Criacional

Problema motivador: O ponto de entrada precisava criar o personagem escolhido pelo jogador, e Item precisava instanciar o efeito correto a partir de uma chave textual. Sem fábrica, ambos dependeriam de todas as classes concretas.

Implementação: PersonagemFactory cria personagens jogáveis; EffectFactory cria efeitos a partir de chaves textuais. O código cliente depende apenas das abstrações retornadas.

```
public class PersonagemFactory {  
    private PersonagemFactory() {}  
    public static Personagem criar(int opcao) {  
        switch (opcao) {  
            case 1: return new Clary();  
            case 2: return new Jace();  
            case 3: return new Isabelle();  
            case 4: return new Alec();  
            default:  
                throw new IllegalArgumentException("Opção de personagem  
inválida: " + opcao);  
        }  
    }  
}
```

```
public class EffectFactory {  
    private EffectFactory() {}  
    public static ItemEffect criar(String chave) {  
        switch (chave.toLowerCase()) {  
            case "cura": return new CuraEffect();  
        }  
    }  
}
```

```

        case "cura_grande": return new CuraGrandeEffect();

        case "forca":

        case "força": return new ForcaEffect();

        case "elixir_forca":

        case "elixir_força": return new ElixirForcaEffect();

        case "defesa": return new DefesaEffect();

        case "bonus_defesa": return new BonusDefesaEffect();

        default:

                                throw new IllegalArgumentException("Efeito
desconhecido: " + chave);
    }
}
}

```

Benefício: ShadowhuntersRPG e Item dependem de abstrações, não de concretos. A criação fica em um único ponto previsível. Custo: O switch não desaparece — ele migra para a fábrica, mas fica isolado.

6.3 Prototype — Categoria: Criacional

Problema motivador: Ao transferir itens de um inimigo derrotado para o jogador, uma cópia rasa compartilharia os mesmos objetos Item, podendo causar alterações acidentais de estado.

Implementação: Personagem, Inventario e Item implementam Cloneable e possuem construtores de cópia, garantindo deep copy em toda a hierarquia.

```

// Personagem — construtor de cópia profundo
protected Personagem(String nome, int pontosVida, int ataque, int
defesa, int nivel) {

    this.nome = nome;

    this.pontosVida = pontosVida;

    this.ataque = ataque;

```

```
        this.defesa = defesa;

        this.nivel = nivel;

        this.inventario = new Inventario(); // deep copy
    }
```

```
// Inventario - clona cada item

public Inventario(Inventario modelo) {

    if (modelo == null) throw new IllegalArgumentException("Modelo
de inventário ausente");

    for (Item item : modelo.itens) {

        itens.add((Item) item.clone()); // cada item é copiado
    }

}
```

```
// Item - copia todos os campos

public Item(Item modelo) {

    if (modelo == null) throw new IllegalArgumentException("Modelo
de item ausente");

    this.nome = modelo.nome;

    this.descricao = modelo.descricao;

    this.efeito = modelo.efeito; / efeitos sem estado: compartilhado
é seguro

    this.quantidade = modelo.quantidade;

}
```

Benefício: Cada cópia é independente; alterar a quantidade de um item no clone não afeta o original.

Custo: Todo campo mutável adicionado deve ser incluído no construtor de cópia.

6.4 Template Method — Categoria: Comportamental

Problema motivador: Todos os personagens compartilham o contrato de habilidade especial (recebe dado e inimigo, não retorna), mas cada um tem uma regra completamente diferente de execução.

Implementação: Personagem define habilidadeEspecial como método abstrato. Cada subclasse implementa sua variação sem alterar o contrato.

```
// Template: contrato fixo na classe abstrata
public abstract class Personagem implements Cloneable {
    public abstract void habilidadeEspecial(Random dado, Inimigo inimigo);
}

// Alec: tiro preciso com 70% de acerto
public class Alec extends Personagem implements Comparable<Alec> {
    private static final int CHANCE_ACERTO_PERCENTUAL = 70;

    @Override
    public void habilidadeEspecial(Random dado, Inimigo inimigo) {
        System.out.println("\n" + nome + " mira cuidadosamente...");
        if (dado.nextInt(100) < CHANCE_ACERTO_PERCENTUAL) {
            int dano = Math.max(0, ataque * 2 - inimigo.getDefesa());
            inimigo.receberDano(dano);

            System.out.println(nome + " acerta! Tiro preciso causou " +
dano + " de dano!");
        } else {
            System.out.println(nome + " errou o alvo desta vez.");
        }
    }
}
```

```
// Clary: runa aleatória — cura ou força
public class Clary extends Personagem implements Comparable<Clary> {
    @Override
```

```

public void habilidadeEspecial(Random dado, Inimigo inimigo) {
    int rolagem = dado.nextInt(6) + 1;

    System.out.println("\n" + nome + " invoca uma Runa
Mística...");

    if (rolagem <= 2) {
        System.out.println("A runa falha em brilhar. Nada
acontece.");
    } else if (rolagem <= 4) {
        pontosVida += 30;

        System.out.println(nome + " ativa a Runa de Criação e
recupera 30 HP!");
    } else {
        ataque += 8;

        System.out.println(nome + " ativa a Runa de Fúria e ganha
+8 de ataque!");
    }
}
}

```

Benefício: Batalha chama sempre jogador.habilidadeEspecial(dado, inimigo) sem saber qual personagem é. O polimorfismo garante o comportamento correto.

Custo: Regras muito diferentes entre subclasses exigem que cada uma carregue sua própria implementação completa.

7. Design de API

O sistema não expõe API pública (REST, GraphQL ou gRPC), pois é uma aplicação de console local sem comunicação em rede. Esta seção documenta as interfaces principais entre os módulos internos.

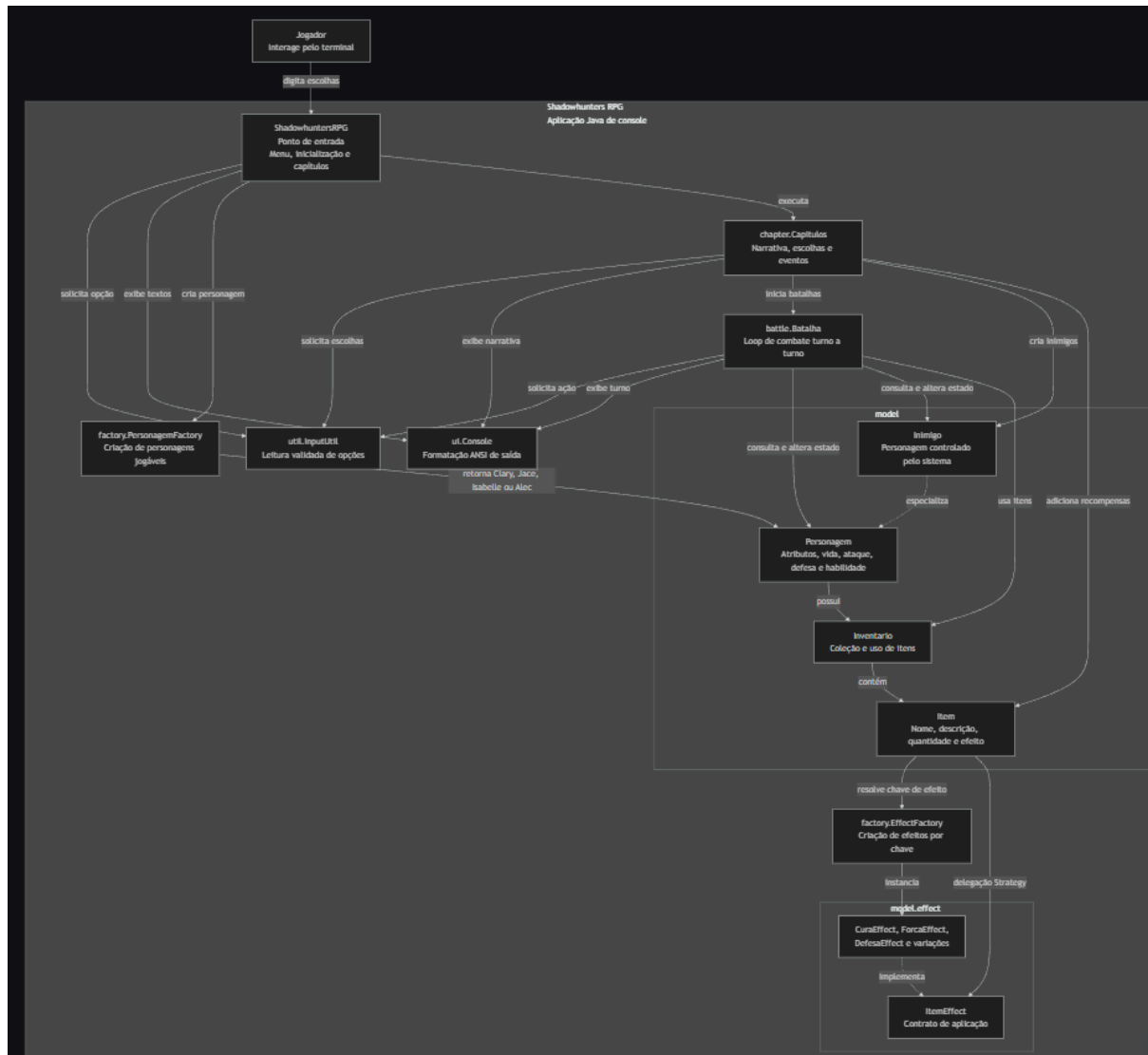
Módulo cliente	Interface	Módulo provedor	Contrato
ShadowhuntersRPG	PersonagemFactory.criar(int)	factory	Retorna Personagem. Lança <code>IllegalArgumentException</code> para opção inválida.
Item	EffectFactory.criar(String)	factory	Retorna <code>ItemEffect</code> . Lança <code>IllegalArgumentException</code> para chave desconhecida
Batalha / Capítulos	<code>InputUtil.lerIntervalo(Scanner, int, int)</code>	util	Retorna int válido no intervalo [min, max]. Nunca retorna valor fora do intervalo.
Batalha	<code>Personagem.habilidadeEspecial(Random, Inimigo)</code>	model	Polimórfico; efeito varia por subclasse. Sem valor de retorno
Inventario	<code>Item.usar(Personagem)</code>	model	Decrementa quantidade e delega a <code>ItemEffect</code> . Lança <code>IllegalStateException</code> se zerado.
Personagem	<code>Inventario.adicionarItem(Item)</code>	model	Empilha itens iguais automaticamente. Clona o item recebido.

Convenções adotadas nas interfaces internas:

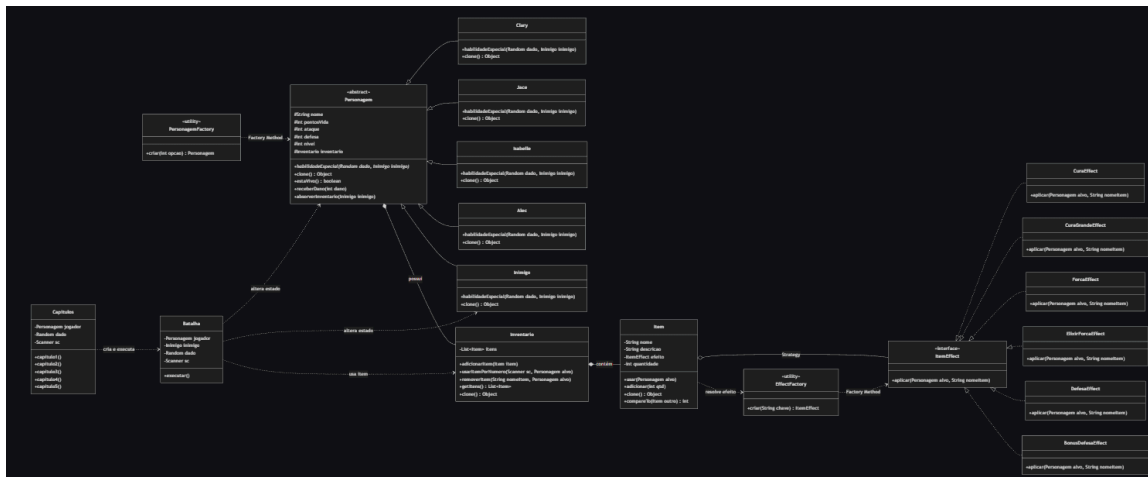
- Erros de pré-condição sinalizam com exceções não verificadas (`IllegalArgumentException`, `IllegalStateException`).
- Métodos que retornam coleções entregam cópias defensivas (`getItens()` retorna `new ArrayList<>(itens)`).
- Toda leitura numérica de entrada passa por `InputUtil.lerIntervalo`, garantindo comportamento único.

8. Diagramas e Modelos

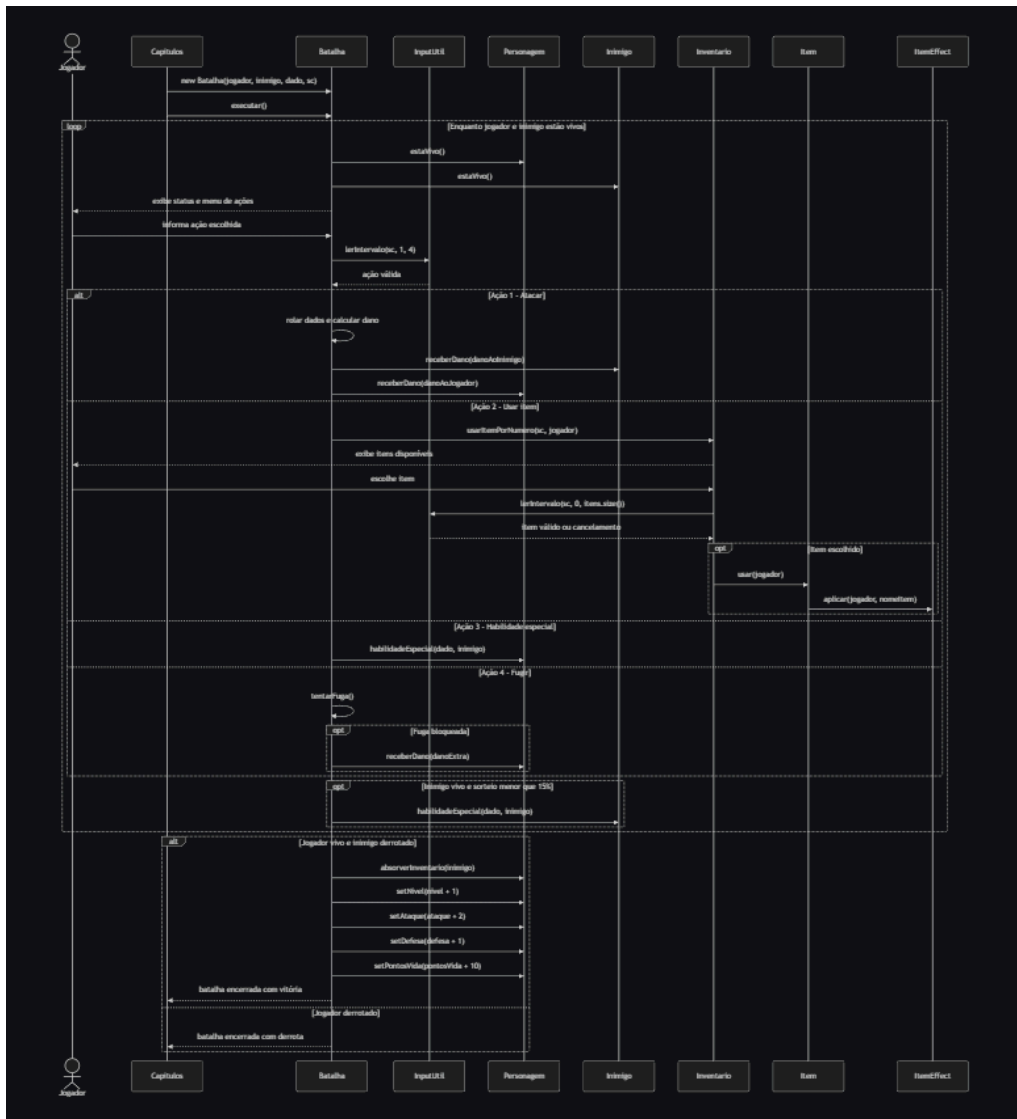
8.1 Arquitetura C4 — Nível de Componentes



8.2 Diagrama de Classes — Padrões GoF



8.3 Diagrama de Sequência — Fluxo de Batalha



9. Conclusões

Resultados Obtidos

A aplicação dos padrões GoF (Strategy, Factory Method, Prototype e Template Method) e dos princípios SOLID resultou em um sistema organizado, extensível e compreensível. A separação por pacotes tornou cada responsabilidade localizável, e as interfaces enxutas facilitaram o entendimento do fluxo de dados entre módulos.

A eliminação de duplicação via InputUtil e Console demonstrou, na prática, como Clean Code reduz o custo de manutenção. O tratamento explícito de erros com exceções significativas tornou o comportamento do sistema previsível mesmo diante de entradas inválidas.

Avaliação Crítica

O padrão Strategy foi o que trouxe maior benefício: transformou um switch crescente dentro de Item em uma hierarquia extensível de classes coesas. O Factory Method complementou isso ao isolar a instanciação do código cliente.

O Prototype é conceitualmente correto, porém exige disciplina: qualquer campo mutável adicionado ao modelo precisa ser propagado para o construtor de cópia, o que pode ser uma fonte de bugs em projetos maiores.

O Template Method, na forma de habilidadeEspecial abstrato, é elegante mas resulta em implementações completamente distintas entre subclasses. Em equipes maiores, um Command pattern poderia ser mais composível.

Limitações e próximas iterações

- Ausência de testes automatizados: a confiabilidade depende de testes manuais. JUnit cobrindo InputUtil, Inventario e cada ItemEffect seria a primeira prioridade em uma próxima iteração.

- Estado global implícito: o Scanner e o Random são passados como parâmetros através de múltiplas camadas. Encapsulá-los em um contexto de jogo reduziria o acoplamento de assinatura.
- Persistência: o jogo não salva o progresso. Uma próxima versão poderia serializar o estado do personagem em JSON para permitir save/load.
- Escalabilidade da narrativa: Capítulos concentra muitos fluxos narrativos. Um padrão Command ou Chain of Responsibility poderia tornar os capítulos plugáveis.
- Internacionalização: mensagens hardcoded limitam a portabilidade do jogo para outros idiomas.

10. Referências Bibliográficas

Livro-texto da disciplina.

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. **Design Patterns: Elements of Reusable Object-Oriented Software**. Reading: Addison-Wesley, 1994.

MARTIN, Robert C. **Clean Code: A Handbook of Agile Software Craftsmanship**. Upper Saddle River: Prentice Hall, 2008.

MARTIN, Robert C. **Clean Architecture: A Craftsman's Guide to Software Structure and Design**. Upper Saddle River: Prentice Hall, 2017.

FOWLER, Martin. **Patterns of Enterprise Application Architecture**. Boston: Addison-Wesley, 2002.

ISO/IEC. **ISO/IEC 25010:2023 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model**. Genebra: ISO, 2023.

NYGARD, Michael T. **Documenting Architecture Decisions**. Disponível em: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>. Acesso em: jun. 2026.

COCKBURN, Alistair. **Hexagonal Architecture**. Disponível em: <https://alistair.cockburn.us/hexagonal-architecture>. Acesso em: jun. 2026.

BLOCH, Joshua. **Effective Java**. 3. ed. Boston: Addison-Wesley Professional, 2018. ISBN 978-0-13-468599-1.

FREEMAN, Eric; ROBSON, Elisabeth. **Head First Design Patterns: Building Extensible and Maintainable Object-Oriented Software**. 2. ed. Sebastopol: O'Reilly Media, 2021. ISBN 978-1-49-207800-5.

ORACLE. **The Java Tutorials: Object-Oriented Programming Concepts**. Disponível em: <https://docs.oracle.com/javase/tutorial/java/concepts/index.html>. Acesso em: jun. 2026.